

Practitioner Craft — LangChain in Production

Track 16.3 – [LangChain](#) · [16-ai-tech-stack/tracks/langchain/notes/02-practitioner-craft.md](#)

Read [01-core-concepts.md](#) first. This note is the operational half: memory, streaming, caching, debugging, cost, and the decision to graduate to LangGraph. Verified against **langchain 1.4.0** / **langchain-core 1.6.3**.

1. Memory: the part that aged worst

v0's `ConversationBufferMemory`, `ConversationSummaryMemory` and `RunnableWithMessageHistory` are no longer in `langchain` — they moved to `langchain-classic`. In v1 the answer is **LangGraph checkpoint state**: give `create_agent` a `checkpoint` and pass a `thread_id`, and the message list persists across turns automatically.

```
from langchain.agents import create_agent
from langgraph.checkpoint.memory import InMemorySaver      # PostgresSaver in prod

agent = create_agent(model="claude-sonnet-4-6", tools=[...], checkpointer=InMemorySaver())
agent.invoke({"messages": [{"role": "user", "content": "hi"}]},
             config={"configurable": {"thread_id": "user-42-session-9"}})
```

What has *not* changed is the hard part: chat history is an unbounded list feeding a bounded context window. You need a policy, and every policy is a trade.

Strategy	How	What it costs you
Keep everything	do nothing	works until it doesn't, then fails at 3am at maximum token price
Trim	<code>trim_messages(msgs, max_tokens=n, strategy="last", token_counter=model)</code>	cheap; loses early facts <i>silently</i>
Summarise	<code>SummarizationMiddleware</code> on <code>create_agent</code>	an extra model call per compaction; errors compound
Extract facts to a store	your own code + a DB, queried per turn	most work; the only one that scales past a long session

The four pitfalls, in the order you will meet them

- Trimming through a tool call.** Drop an `AIMessage` carrying `tool_calls` but keep its `ToolMessage` (or the reverse) and the provider rejects the whole conversation. `trim_messages` has `start_on="human"` and `include_system=True` precisely for this. Use both.
- Summaries that drift.** Each summarisation summarises the previous summary; specifics decay into generalities and then into fiction. Keep the last N turns verbatim *alongside* the summary.

3. **thread_id** is a security boundary. A collision leaks one user's conversation to another. Derive it from an authenticated user id server-side. Never from a value the client sends.
4. **Silent cost growth.** Untrimmed history means every turn re-sends the whole conversation. Turn 30 of a chat can cost 30x turn 1. Log input tokens per turn and alert on the slope, not the total.

Opinion: default to *trim + a short pinned summary of durable facts*. Full summarisation is attractive on a slide and lossy in practice; pure trimming is honest about what it throws away.

2. Streaming: measure time-to-first-token or don't bother

`.stream()` yields output chunks. `.astream_events()` yields a typed event per nested runnable — `on_chain_start`, `on_prompt_end`, `on_chat_model_stream`, `on_tool_end`, `on_parser_end` — each with a `run_id` and `parent_ids`. That second API is what you debug with, and it is structurally the same data LangSmith ships over the wire.

The gotcha that silently kills streaming: a chain streams only if *every* step streams.

Step	Streams?
<code>ChatPromptTemplate</code>	n/a (runs once, up front)
<code>ChatModel</code>	yes, natively
<code>StrOutputParser</code>	yes — passes chunks through
<code>JsonOutputParser</code>	partially — emits progressively parseable prefixes
<code>RunnableLambda</code> over a full string	no — buffers the entire stream, then emits one chunk
a retriever	no — it blocks; there is nothing to stream until it returns

`lcel_basics.py` demo 2 proves this: appending one innocent `RunnableLambda(str.upper)` turns a 10-chunk stream into a 1-chunk stream with no warning. If you have a "streaming" endpoint, assert on chunk count in a test, because nothing else will tell you.

For a RAG chain, time-to-first-token includes embed + vector search. That, not total generation time, is the number your users feel and the number to optimise.

3. Callbacks and token accounting

`BaseCallbackHandler` is the extension point everything else is built on — `on_llm_start`, `on_llm_new_token`, `on_tool_error`, `on_chain_end`. Pass handlers per call:

```
chain.invoke(x, config={"callbacks": [MyHandler()], "run_name": "faq", "tags": ["prod"]})
```

Two rules: - **Async chains need async handlers**. A sync handler inside an async chain blocks the event loop and turns your concurrent server into a serial one. - **A handler that raises can take down the run**. Wrap handler bodies in try/except; a metrics failure should never be a user-facing failure.

Token accounting. `AIMessage.usage_metadata` carries `input_tokens`, `output_tokens`, `total_tokens`, plus `input_token_details.cache_read` and `cache_creation` when provider prompt caching is in play. To aggregate across many calls:

```
from langchain_core.callbacks import get_usage_metadata_callback

with get_usage_metadata_callback() as cb:
    ...
    print(cb.usage_metadata)  # {model_name: {input_tokens: ..., output_tokens: ...}}
```

Log it per request with a tenant id. Cost surprises are almost always distributional — one customer, one prompt shape, one retry loop — and an aggregate dashboard hides exactly that.

4. Caching: two different things with the same name

	<code>set_llm_cache(InMemoryCache())</code>	Provider prompt caching
Where	your process (or SQLite / Redis)	the provider's servers
Keyed on	exact prompt + model + params	a prompt prefix
Hit rate in prod	~0 (prompts are unique)	high, if your system prompt is stable
Saves	the whole call	input-token cost on the cached prefix
Enable with	<code>langchain_core.globals.set_llm_cache</code>	<code>AnthropicPromptCachingMiddleware</code> , <code>BedrockPromptCachingMiddleware</code>

Exact-match caching is *excellent* in tests and eval loops, where you run the same 200 prompts fifty times. Do not put it in a request path and expect savings. Provider prompt caching is the one that cuts the bill; it needs a long, stable prefix and usually has a minimum-token threshold.

5. The debugging playbook

In order of what to reach for:

1. **LangSmith.** `export LANGSMITH_TRACING=true`, `export LANGSMITH_API_KEY=...`. That is the whole integration — no code change. Every runnable becomes a span with inputs, outputs, latency, tokens and errors. → `../langsmith/`
2. `astream_events()` when you want the same tree locally, in code, in a test assertion.

3. `set_debug(True)` / `set_verbose(True)` from `langchain_core.globals` — loud, unstructured, fine for a 30-second question, useless in a long-running service.
4. `chain.get_graph().print_ascii()` to see the pipeline you actually built (needs `pip install grandalf`).
5. **Name your runs.** `.with_config(run_name="...")` on every non-trivial sub-chain. An unnamed trace is forty spans called `RunnableSequence`, and it is the reason people say traces don't help.

Failure modes and what they actually mean

Symptom	Real cause	Fix
<code>KeyError: 'foo'</code> from a prompt	unescaped <code>{</code> in a literal, or a <code>RunnableParallel</code> key mismatch	escape <code>{{</code> ; check <code>chain.input_schema</code>
Stream returns one chunk	a buffering step mid-chain	move the lambda to the end, or after the response
"tool_use ids must have tool_result"	a missing <code>ToolMessage</code> — usually from trimming	<code>start_on="human"</code> ; answer every <code>tool_call_id</code>
Model calls the wrong tool	vague docstring / overlapping tool descriptions	rewrite the docstring; it is the prompt
<code>ValidationError</code> from <code>with_structured_output</code>	model ignored a pydantic constraint	<code>include_raw=True</code> , then retry with the error in context
Cost 3x the estimate	retries on non-retryable errors, or untrimmed history	constrain <code>retry_if_exception_type</code> ; trim
<code>ImportError</code> on a v0 symbol	you are on a pre-1.0 tutorial	consult the v1 migration guide, or <code>pip install langchain-classic</code>
Latency spikes at p99 only	<code>with_retry</code> exponential backoff firing on a flaky provider	cap <code>stop_after_attempt</code> ; add a fallback model

6. Cost control, concretely

1. **Route by difficulty.** Cheap model first; escalate on low confidence or an explicit "I'm not sure". `with_fallbacks` gives you the mechanism; the routing decision is yours.
2. **Cap output tokens.** Output is the expensive half on every provider. `max_tokens` is a budget, not a safety net — set it.
3. **Retry only what is retryable.** `retry_if_exception_type=(RateLimitError, APIConnectionError)`. A blanket `.with_retry()` pays three times for the same malformed prompt.
4. **Trim history.** See §1. This is usually the largest single line item in a chat product.
5. **Batch offline work.** `chain.batch(inputs, config={"max_concurrency": 8})` for backfills and evals.
6. **Cache in evals.** `set_llm_cache(SQLiteCache("evals.db"))` makes an eval re-run nearly free.
7. **Attribute cost per tenant** from `usage_metadata`, not from the provider's monthly invoice.

7. Testing a LangChain app

This is the part most teams skip and then regret.

- **Fake the model.** `langchain_core.language_models.fake_chat_models` ships `FakeListChatModel`, `GenericFakeChatModel` and `ParrotFakeChatModel`; `../code/fakes.py` shows how to roll your own in ~40 lines. A scripted model lets you test the *loop* — "does it recover when the tool fails?" — deterministically, in milliseconds, for free.
 - **Assert on structure, not prose.** Test that a `ToolMessage` exists for every `tool_call_id`, that the stream produced >1 chunk, that the retriever was called once. Do not assert on model wording.
 - **Golden traces.** Record `astream_events` output for a known input; diff it after a refactor. This catches "the retriever silently stopped being called" faster than any output assertion.
 - **Contract-test your tools independently** of the model. They are just functions.
 - **Then** do real evals against a real model — that is `../../15-ai-evals/`.
-

8. Security notes specific to this framework

- **Retrieved documents are untrusted input.** RAG splices attacker-controllable text into your system prompt. If your agent has tools, a poisoned document is a remote tool-call primitive. → `../../13-ai-security/`
 - **Tool arguments come from a language model.** Validate them like any other user input: parameterise SQL, allowlist paths, bound numeric ranges. `args_schema` gives you types, not authorisation.
 - **langchain-community is community-maintained.** Some integrations shell out, evaluate expressions, or hit arbitrary URLs. Read the source of any community tool before it reaches production.
 - **Tenant isolation lives in the vector-store filter,** enforced server-side. A prompt that politely asks the model to only use tenant A's documents is not an access control.
 - **Secrets in traces.** LangSmith captures inputs and outputs verbatim. Redact before you trace, or configure hiding — otherwise your API keys and PII end up in a SaaS trace store.
-

9. When to graduate to LangGraph

Move when **any** of these is true:

- there is a **loop** — model → tool → model → ... — whose length you cannot know in advance;
- you need **durable execution**: resume mid-workflow after a crash or a deploy;
- you need **human-in-the-loop**: pause, get approval, continue;
- you need **per-thread persistence** across HTTP requests;
- you have **multiple agents** handing work to each other;
- you want to **stream intermediate state**, not just final tokens.

It is not a rewrite. Prompts, models, tools, retrievers and parsers are already `Runnable`s, and `LangGraph` nodes take `Runnable`s. What changes is control flow: a pipe becomes a graph with state. `create_agent` in `langchain.agents` is the prebuilt front door and returns a compiled `LangGraph` graph.

Signals you graduated too early: your graph has three nodes in a straight line and no conditional edges. That is a chain with extra ceremony — write it in LCEL.

Signals you graduated too late: you have a `while True:` around `chain.invoke`, a hand-rolled dict of conversation state keyed by session id, and a `try/except` that restarts the loop from the top. You have written a worse `LangGraph`. → `../langgraph/`

10. A pre-ship checklist

- [] Every non-trivial sub-chain has a `run_name`.
- [] Tracing is on in staging, and secrets are redacted before they reach it.
- [] `with_retry` is constrained to transient exception types, with a bounded attempt count.
- [] Every model call has a fallback, or a documented decision not to have one.
- [] History has a trimming policy, and `start_on="human"` so tool pairs survive it.
- [] Every tool has a `max` execution timeout and an error path that returns a `ToolMessage`.
- [] The agent loop has a hard turn limit.
- [] Structured output is wrapped in a `ValidationError` handler.
- [] `usage_metadata` is logged per request with a tenant id.
- [] Vector-store queries are filtered by tenant, server-side.
- [] Dependencies are pinned to a major (`langchain-core>=1.6,<2`), and you have read the changelog.
- [] There is a test that runs the whole pipeline against a fake model, offline, in CI.